In the above example of the newly available '->' operator, you can observe that after the first match is found, no further checks are done. Similarly, now the switch statements can also return values that simplify the syntax, as follows:

```
public class MyClass {
 public static void main(String args[]) {

  int n = 1;
  String value = switch (n) {
    case 1 -> "One";
    case 2 -> "Two";
    default -> "NaN";
   };
  System.out.println(value);
 }
}
```

The above code will return 'One' and assign it to the variable 'value'.

JEP 341: Default CDS archives

Before the bytecode from a class is executed, there are a few preparatory steps that take place. The class is loaded, by name, by the class loader, and by verification of the bytecode done. Then it's loaded in an internal data structure. This process takes some time and if there are a large number of classes that are loaded, the JVM startup time will get affected.

As long as the classes in the *jar* are not changed, this process is redundant when you launch multiple applications or the same application many times. Here is where class data sharing (CDS) comes in. It allows you to create the data structures once, dump them in a file and reload again. This is not a new feature and was actually released in JDK 5. Previously, the application had to be started with the addition argument *'-Xshare:on'* to enable it. With Java 12, this feature is enabled by default.

To measure the benefits running a 'HelloWorld' program with CDS off, takes ~0.36s, and with CDS on, takes ~0.28s on my machine. This is around 80ms for a 'Hello World' program; so for larger applications, this feature could increase the startup time. Also, by passing some additional arguments, this can be extended for application class files using a feature called AppCDS.

JEP 189: Shenandoah - an experimental low-pause-time garbage collector

This is a new experimental garbage collector supported by Red Hat for aarch64 and amd64. Shenandoah aims to provide predictable and short GC pauses independent of the heap size. So, irrespective of whether your heap size is 200MB or 200GB, Shenandoah will have the same consistent pause times.

Shenandoah trades concurrent CPU cycles and space for pause time improvements. It contains an indirection pointer to every Java object, which enables the GC threads to compact the heap while the Java threads are running. Marking and compacting are performed concurrently; so we only need to pause the Java threads long enough to scan the thread stacks in order to find and update the roots of the object graph.

In modern computers, we have more and more CPU and memory. On the one hand, applications are getting bigger and more complex, while on the other hand the SLA guarantees a faster response time. The goal for Shenandoah is to allow programs to run in the available memory, and also be optimised to never interrupt the running program for more than a few milliseconds.

JEP 344: Abortable mixed collections for G1

Previously, G1 used an advanced analysis engine to select the amount of work to be done during a collection (this is partly based on the application's behaviour). The result of this selection is a set of regions called the collection set. Once the collection set has been determined and the collection has been started, the G1 must collect all live objects in all regions of the collection set without stopping. This behaviour can lead to G1 exceeding the pause time goal if the heuristics choose a too-large collection set. This feature intends to make G1 mixed collections abortable if they exceed the pause target.

JEP 346: Promptly return unused committed memory from G1

Prior to this enhancement, G1 only returned memory from the Java heap to the operating system at either a full GC or during a concurrent cycle. In JDK 12, G1 will periodically try to trigger a concurrent cycle to find the overall heap usage, and return unused memory to the OS in a more predictable and periodic manner. This is useful in cloud scenarios where the billing is also done on the amount of memory consumed or required by the application. An additional flag *--XX:G1PeriodicGCInterval* — has also been added to set the interval between checks manually.

These are some of the new features released in Java 12. You can explore all the enhancements at *https://openjdk.java.net/projects/jdk/12/*. END

By: Shiva Saxena

The author is a FOSS enthusiast, currently working on IoT and cloud technologies at SAP. He can be reached at *shivasaxena@outlook.com*. The views expressed in the article are solely his, and do not represent the company's views.